

Industrial Oriented Mini Project Report (CM653PC)
on
“Emergency Phone Recovery Framework”

Submitted
In the partial fulfillment of the requirements for
the award of the degree of

Bachelor of Technology

In

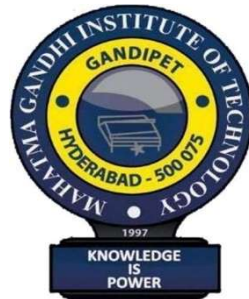
Computer Science & Engineering
(Artificial Intelligence and Machine Learning)

by

A.PRABAHAS
(23261A6602)

Under the guidance of

Prof. M. Rama Bai
(Professor & HoD)



Department of Emerging Technologies

MAHATMA GANDHI INSTITUTE OF TECHNOLOGY(Autonomous)

(Affiliated to Jawaharlal Nehru Institute of Technological University Hyderabad)

Kokapet(V), Gandipet(M), Hyderabad. Telangana – 500 075

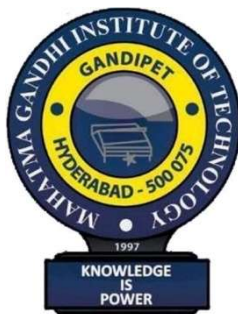
AY 2025-2026

MAHATMA GANDHI INSTITUTE OF TECHNOLOGY(Autonomous)

(Affiliated to Jawaharlal Nehru Technological University Hyderabad)

Gandipet, Hyderabad- 500 075, Telangana (India)

CERTIFICATE



This is to certify that the Industrial Oriented Mini Project (CM653PC) entitled “**Emergency Phone Recovery Framework**”, is being submitted by **Mr. ANUMULA PRABHAS** bearing **Roll No: 23261A6602** in partial fulfilment for the award of degree of **Bachelor of Technology in Computer Science & Engineering – (AI and ML)** to **Mahatma Gandhi Institute of Technology** is a record of bonafide work carried out by her under my guidance and supervision. The results embodied in this project have not been submitted to any other University or Institute for the award of any degree or diploma.

PROJECT GUIDE

HEAD OF THE DEPARTMENT

Dr.M. Rama Bai

Professor, Dept of ET

EXTERNAL EXAMINER

DECLARATION

This is to certify that the work reported in this project titled “**Emergency Phone Recovery Framework**” is a record of work done by me in the Department of Emerging Technologies, Mahatma Gandhi Institute of Technology, Hyderabad.

No part of the work is copied from books/journals/internet and wherever the portion is taken, the same has been duly referred to in the text. The report is based on the work done entirely by me and not copied from any other source.

A. PRABHAS
23261A6602

ACKNOWLEDGEMENT

The satisfaction that accompanies the successful completion of any task would be incomplete without the mention of people who made it possible because success is the abstract of hardwork and perseverance, but steadfast of all is encouraging guidance. So, I acknowledge all those whose guidance and encouragement served as a beacon light and crowned my efforts with success.

I would like to express my sincere thanks to, **Prof G. ChandraMohan Reddy, Principal MGIT**, for providing the working facilities in college.

I am extremely thankful and indebted to my head and mentor **Prof. M. Rama Bai, Professor and HoD** Department of Emerging Technologies, for her constant guidance, encouragement and moral support throughout the project.

I am extremely thankful to **Ms. J. Sreedevi, Assistant Professor and Ms. Aasha Jyothi, Department of ET, MGIT**, Industrial Oriented Mini Project Coordinators for their encouragement and support throughout the project.

Finally, I would also like to thank all the faculty and staff of the ET Department who helped me directly or indirectly in completing this project.

A.PRABHAS
23261A6602

TABLE OF CONTENTS

	PAGE NO
Certificate	i
Declaration	ii
Acknowledgement	iii
List of Figures	v
List of Tables	vi
Abstract	vii
1. INTRODUCTION	1
1.1 Problem Statement	
1.2 Existing System	
1.3 Proposed System	
1.4 Project Scope & Objectives	
1.5 System Requirements	
2. LITERATURE SURVEY OF EMERGENCY PHONE RECOVERY FRAMEWORK	4
3. DESIGN AND METHODOLOGY OF EMERGENCY PHONE RECOVERY FRAMEWORK	8
3.1 System Architecture	
3.2 Use Case Diagram	
3.3 Required Modules/Libraries/Framework	
3.4 Methodology and Process	
4. RESULTS	12
4.12 Test Cases and Validation	
5. CONCLUSION AND FUTURE SCOPE	19
BIBLIOGRAPHY	20
APPENDIX	21

LIST OF FIGURE

FIGURE NO	FIGURE NAME	PAGE NO
Figure 3.1	System Architecture Flowchart	8
Figure 3.2	Use case diagram	9
Figure 4.1	App Lock Setup Interface	12
Figure 4.2	Recovery Password Setup Interface	13
Figure 4.3	Trusted Email Setup Interface	13
Figure 4.4	Physical Recovery Key	14
Figure 4.5	Firebase Realtime Database	14
Figure 4.6	Shutdown Verification screen	15
Figure 4.7	Normal Mode Dashboard	15
Figure 4.8	Stolen Mode Interface	16
Figure 4.9	Live Location Tracking System	16
Figure 4.10	Expired Recovery System	17
Figure 4.11	Recovery Mode Interface	17

LIST OF TABLES

TABLE NO.	TABLE NAME	PAGE NO
Table 2.1	Literature Survey Table	6
Table 4.1	Test Case Table	18

ABSTRACT

The Emergency Phone Recovery Framework is an Android-based smartphone security and recovery system designed to address the growing issues of mobile theft, unauthorized access, and personal data loss. Due to operating system restrictions, implementing complete device-level recovery mechanisms through third-party applications is challenging. To overcome these limitations, the proposed framework simulates smartphone-level security features within an application environment. Developed using Android Studio and Kotlin, the framework integrates Firebase Authentication and Firebase Realtime Database for secure user authentication, cloud synchronization, data storage, and emergency alert management. The system operates through three modes: Normal Mode, Stolen Mode, and Recovery Mode. In Normal Mode, users configure security settings such as application lock passwords, recovery passwords, and trusted email addresses, which are securely synchronized with Firebase. The framework continuously monitors unauthorized shutdown attempts by verifying the screen lock password. If incorrect credentials are entered repeatedly beyond the allowed limit, the system identifies suspicious activity and automatically activates Stolen Mode. During this phase, device access is restricted, live location tracking is enabled, activity logs are recorded, and emergency alerts are sent to the registered owner. All security events and recovery-related information are stored in Firebase for real-time monitoring and recovery support. The Recovery Mode uses a secure recovery password verification mechanism that allows only the legitimate owner to regain access. Upon successful verification, the framework restores normal operation, demonstrating the practical integration of Android development, cloud services, and smartphone security concepts in a lightweight and user-friendly solution.

1.INTRODUCTION

1.1 PROBLEM STATEMENT

Smartphone theft has become a major security concern due to the increasing amount of personal, financial, and sensitive information stored on mobile devices. In many theft situations, the thief immediately powers off the phone or removes internet access, making conventional tracking systems ineffective. Existing mobile recovery solutions mainly depend on reactive approaches such as location tracking after theft, which often fail once the device is shut down. Additionally, operating system-level restrictions limit third-party applications from implementing complete phone-level recovery mechanisms. There is a need for a proactive security framework that can respond instantly to suspicious activities before the device becomes inaccessible. The proposed Emergency Phone Recovery Framework addresses this issue by modeling the application as a virtual phone-level control system capable of activating emergency protection, restricting unauthorized access, verifying ownership, and assisting the user in recovering the device efficiently

1.2 EXISTING SYSTEM

Google Find My Device : Existing recovery systems such as Find My Device mainly focus on tracking lost Android devices using GPS, internet connectivity, and Google account synchronization. The system allows users to locate, ring, or erase the device remotely.

Samsung SmartThings Find : Samsung SmartThings Find provides device tracking and offline finding support for Samsung devices through connected Samsung networks. It helps users locate nearby lost devices within the ecosystem.

Apple Find My : Find My offers location tracking, lost mode, and remote locking features for Apple devices using iCloud synchronization and connected device services.

Third-Party Anti-Theft Applications: Many anti-theft applications provide alarm systems, remote wipe options, and basic recovery support. However, most applications depend on internet connectivity, GPS availability, and active device status for proper operation.

Drawbacks of the Existing System:

- No proactive theft prevention mechanism.
- Existing systems allow unauthorized phone shutdowns.
- Tracking fails immediately after phone shutdown.
- Phones becomes difficult or impossible to locate after power off.
- Lack of layered security and recovery verification mechanisms.
- Existing solutions react late

1.3 PROPOSED SYSTEM

The proposed “Emergency Phone Recovery Framework” is an Android-based security and recovery solution designed to provide proactive smartphone protection during theft situations. Unlike existing systems that react only after a device is lost, the proposed framework attempts to detect suspicious shutdown activities before the phone becomes inaccessible. The application is conceptually designed as a virtual phone-level control system that simulates advanced security and recovery mechanisms within an Android application environment.

The framework operates using three major modes: Normal Mode, Stolen Mode, and Recovery Mode. In Normal Mode, the application functions like a secure mobile environment for authorized users. When suspicious activities such as unauthorized shutdown attempts are detected, the system instantly activates Stolen Mode, which restricts access and prevents unauthorized operation of the mobile device. Simultaneously, alerts and recovery-related information are synchronized through Firebase cloud services.

Recovery Mode acts as a secure Recovery Gate where only the legitimate owner can regain access using a recovery password verification mechanism. This layered security approach improves recovery chances and provides proactive protection against smartphone theft.

1.4 PROJECT SCOPE & OBJECTIVES

PROJECT SCOPE

- Propose an Emergency Phone Recovery Framework for smartphone security and recovery assistance.
- Model the application as a virtual phone-level control system to demonstrate the core recovery concept.
- Implement Normal Mode, Stolen Mode, and Recovery Mode for different security operations.
- Detect suspicious shutdown attempts and activate emergency protection mechanisms instantly.
- Restrict unauthorized access during theft situations using Stolen Mode protection.
- Demonstrate cloud synchronization, owner verification, and emergency alert mechanisms using Firebase services.

OBJECTIVES

- Provide proactive protection against smartphone theft and unauthorized access.
- Simulate phone-level security and recovery mechanisms through an application-based framework.
- Detect suspicious shutdown activities before the device becomes inaccessible.
- Restrict thieves from operating the mobile device after unauthorized activity detection.

1.5 SYSTEM REQUIREMENTS

Software Requirements

- **Operating System:** Windows 10/11 used for application development and framework testing
- **Android Studio:** Used as the primary development environment for implementing and demonstrating the framework prototype.
- **Kotlin:** Primary programming language used for Android application development.
- **Firebase Authentication:** Used for secure user authentication and login management.
- **Firebase Realtime Database:** Used for cloud synchronization, recovery information storage, and emergency data management.
- **Google APIs / GPS Services:** Used for location tracking and recovery-related functionalities
- **fpdf2:** PDF generation library for downloadable scan reports.
- **Android SDK:** Required for Android application development, emulator support, and device testing.
- **XML :** Used for designing application user interfaces and layouts.
- **Internet Connectivity:** Required for Firebase synchronization, email alerts, and recovery operations.

Hardware Requirements

- **Laptop/ Computer :** System with Intel i3/i5 processor or equivalent for development and testing purposes.
- **RAM:** Minimum 4 GB RAM; 8 GB recommended for smooth Android Studio performance.
- **GPU (Optional):** NVIDIA GPU with CUDA support for faster model training.
- **Storage:** Minimum 10 GB free storage space for Android Studio, SDKs, emulator files, and project data.
- **Smartphone Device :** Android smartphone with Android version 13 or above used for implementing and testing the Emergency Phone Recovery Framework , with basic sensors like, GPS, accelerometer, and other essential mobile sensors required for detection and recovery operations.

2.LITERATURE SURVEY

[1] A. Parmar, K. Chandravanshi, S. Tiwari, V. D. Kola, M. Kumar, and S. Srivastava, "Anti-theft Mobile Tracking Application," Learning and Analytics in Intelligent Systems (LAIS), Vol. 55, pp. 206–216, 2026.

This study proposes an Android-based anti-theft system that utilizes SIM monitoring, GPS tracking, and emergency SMS alerts for smartphone recovery. The framework enables users to locate stolen devices and identify unauthorized SIM card replacement. By integrating location tracking and notification mechanisms, the system improves the probability of recovering lost devices. However, the solution mainly relies on SIM-based detection and does not provide continuous device lockdown or proactive theft prevention mechanisms.

[2] S. Roy, A. K. Shah, and U. Bhattacharya, "Touch and Track: An Anti-theft and Data Protection Technique for Smartphones," Communications in Computer and Information Science (CCIS), Vol. 625, pp. 347–357, 2016.

This paper presents a smartphone anti-theft framework that combines device tracking and data protection features. The system is capable of identifying theft scenarios and initiating tracking operations while securing user data. The proposed approach improves smartphone security through integrated protection mechanisms. However, practical implementation is limited by Android operating system restrictions and permission constraints.

[3] M. E. Shalini and V. Shanti, "A Novel Location-Based Anti-Theft Image Capture Application with Culprit Identification for Smartphones," TPM – Testing, Psychometrics, Methodology in Applied Psychology, Vol. 32, Issue S5, pp. 286–292, 2025.

This work proposes a location-based smartphone recovery application that captures images of unauthorized users and sends them along with location information to the device owner. The framework improves recovery chances through image-based identification and location tracking. However, the effectiveness of the system depends significantly on network connectivity and cloud service availability.

[4] N. Tasnim, "Smartphone Loss Prevention System Using BLE and GPS Technology," Master's Thesis, The University of Western Ontario, 2023.

This research presents a smartphone loss prevention framework using Bluetooth Low Energy (BLE) and GPS technologies. The system helps users identify and recover misplaced devices before permanent loss occurs. However, recovery effectiveness decreases when BLE communication becomes unavailable..

[5] G. Zheng, H. Zhang, Z. Wang, Y. Wei, and M. Dong, "Mobile Phone Anti-theft Method Based on Mobile Track and User Characteristic," Proceedings of BIGCOM 2019, pp. 28–32, 2019.

This paper proposes an anti-theft approach based on user behavior analysis and mobile trajectory tracking. The system detects suspicious activities and generates alerts during abnormal device usage. However, continuous monitoring requires additional computational resources and battery consumption.

[6] M. Z. Baik, M. M. Alqahtani, and A. Banjar, "Detecting Theft over Pattern Change Recognition and Locating Stolen Micro Mobility Vehicles in Uncovered Signals Areas," Proceedings of ICCIT 2025.

This study introduces theft detection using pattern recognition and location tracking techniques. The framework improves theft identification even in environments with limited signal coverage. However, the solution is primarily designed for mobility assets rather than smartphones.

[7] M. L. Polla, F. Martinelli, and D. Sgandurra, "A Survey on Security for Mobile Devices," IEEE Communications Surveys & Tutorials, Vol. 15, No. 1, pp. 446–471, 2013.

This survey reviews security threats, vulnerabilities, and protection mechanisms for mobile devices. The study discusses authentication, malware detection, and intrusion prevention techniques. However, it does not specifically address smartphone recovery and theft prevention frameworks

[8] Z. Muhammad, Z. Anwar, A. R. Javed, B. Saleem, S. Abbas, and T. R. Gadekallu, "Smartphone Security and Privacy: A Survey on APTs, Sensor-Based Attacks, Side-Channel Attacks, Google Play Attacks, and Defenses," Technologies, Vol. 11, No. 3, Article 76, 2023.

This paper presents a comprehensive survey of smartphone security threats and privacy attacks. The research discusses modern defense mechanisms and attack detection techniques. However, it primarily focuses on cybersecurity threats rather than smartphone theft recovery..

[9] A. Rakhmadi, D. K. Aji, S. Rosad, A. Aziz, and R. Wahyusari, "An IoT-Based Motorcycle Security System Using Arduino and Android for Theft Prevention," Compiler, Vol. 14, No. 1, pp. 43–52, 2025.

This work combines IoT sensors and Android applications for theft prevention and monitoring. The framework demonstrates effective remote security management and alert generation. However, the system is designed for vehicle security and not specifically for smartphone recovery.

[10] J. Landgren and C. Larsson, "Proximity Based Mobile Device Theft Protection," Bachelor's Thesis, Linnaeus University, p. 69, 2024.

This study investigates proximity-based theft prevention using Bluetooth Low Energy technologies. The framework continuously verifies device presence and detects unauthorized separation. However, the system focuses primarily on theft prevention and provides limited support for post-theft recovery operations

Table .2.1 Literature Survey Table of Emergency Phone Recovery Framework

S.no	Title of the Paper	Technology/ Method Used	Advantages	Disadvantages
1	Anti-theft Mobile Tracking Application (2026)	GPS Tracking, SIM Monitoring, SMS Alerts	Real-time tracking	Fails after shutdown
2	Touch and Track: An Anti-theft and Data Protection Technique for Smartphones (2016)	Firebase, Realtime DB	Cloud synchronization	No proactive protection
3	A Novel Location-Based Anti-Theft Image Capture Application with Culprit Identification for Smartphones (2025)	Multi-layer Authentication	Better security	No shutdown prevention
4	Smartphone Loss Prevention System Using BLE and GPS Technology (2023)	GPS APIs	Accurate tracking	No tracking after power off
5	Mobile Phone Anti-theft Method Based on Mobile Track and User Characteristic (2019)	Alarm, SIM Detection	Basic phone security	Reactive approach
6	Detecting Theft over Pattern Change Recognition and Locating Stolen Micro Mobility Vehicles in Uncovered Signals Areas (2025)	Cloud Authentication	Secure verification	Internet dependency

7	A Survey on Security for Mobile Devices (2013)	Activity Detection	Early detection	Limited recovery
8	Smartphone Security and Privacy: A Survey on APTs, Sensor-Based Attacks, Side-Channel Attacks, Google Play Attacks, and Defenses (2023)	Firestore Framework	Remote monitoring	No forced shutdown recovery
9	An IoT-Based Motorcycle Security System Using Arduino and Android for Theft Prevention (2025)	Layered Security	Improved protection	OS limitations
10	Proximity Based Mobile Device Theft Protection (2024)	Virtual Phone Framework	Proactive recovery	Prototype implementation

3. DESIGN AND METHODOLOGY

The proposed Emergency Phone Recovery Framework follows a proactive smartphone security and recovery methodology designed to simulate phone-level protection mechanisms within an Android application environment. The framework is implemented using Android Studio and Kotlin with Firebase used for cloud synchronization, authentication, and emergency alert management.

3.1 SYSTEM ARCHITECTURE

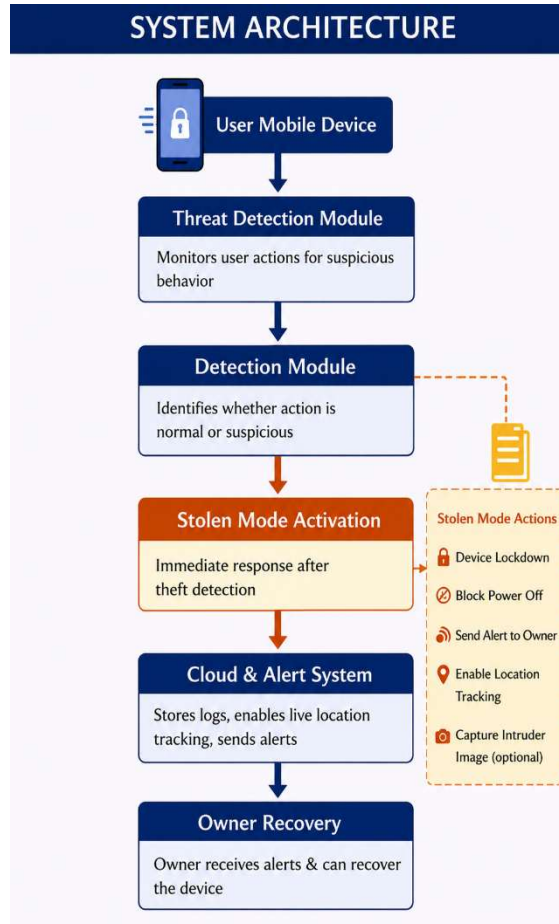


Figure 3.1 Architecture Flowchart of Emergency phone recovery

As shown in the Fig 3.1, the framework begins with the **User Mobile Device**, where the application operates as a virtual phone-level control environment. The detected information is then processed by the **Detection Module**, where the framework analyses whether the detected activity is normal or suspicious. If suspicious behaviour is identified, the system immediately activates **Stolen Mode Activation**. In this stage, emergency protection mechanisms such as device lockdown, shutdown blocking, owner alert generation, and live location tracking are triggered instantly to prevent unauthorized usage of the mobile device.

The framework then communicates with the **Cloud & Alert System**, where Firebase services are

used to store logs, synchronize recovery-related information, and send emergency notifications to the owner. Optional security mechanisms such as intruder image capture can also be integrated for additional protection.

Finally, the system enters the **Owner Recovery stage**, where the legitimate owner receives alerts and can securely recover access to the virtual phone environment using the Recovery Gate and recovery password verification mechanism. The methodology demonstrates how application-level architectures can conceptually model proactive smartphone security and recovery systems despite operating system limitations.

3.2 USE CASE DIAGRAM

The Use Case Diagram illustrates the interaction between the Owner (Actor) and the Emergency Phone Recovery Framework within the system boundary. The Owner performs operations such as Initial Security Setup, Managing Security Settings, Receiving Notifications, Viewing Live Location, Tracking the Device, and Recovering the Device. These actions represent the primary functionalities

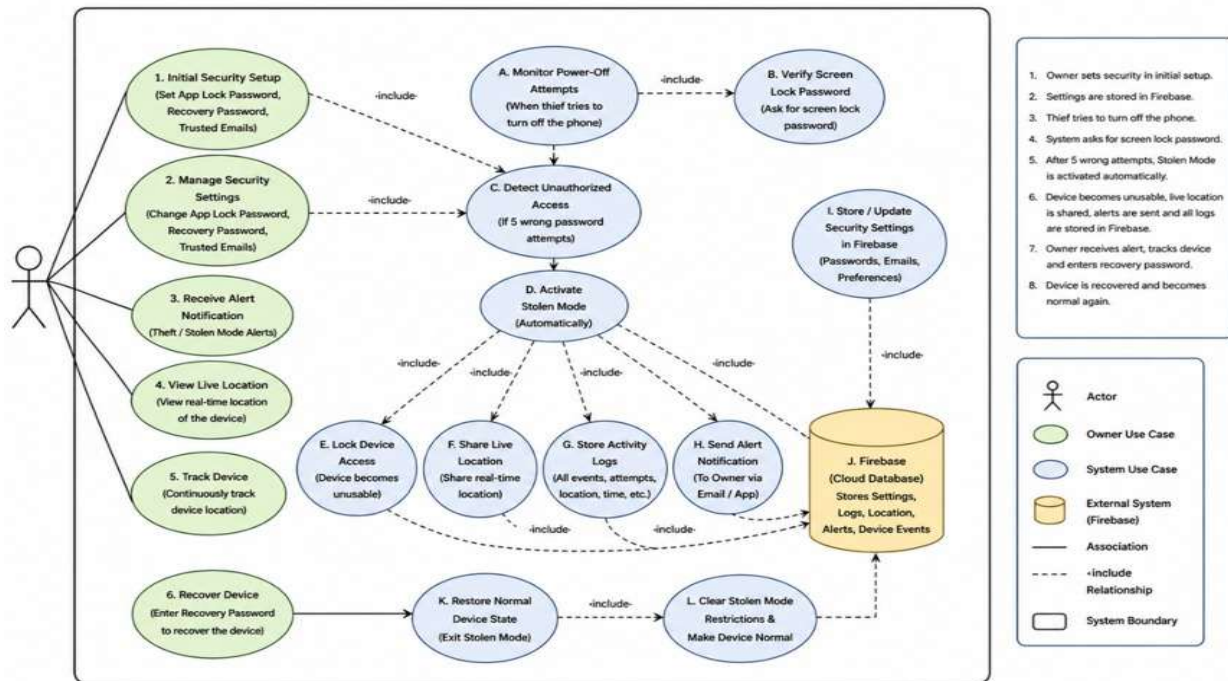


Figure 3.2 Use case Diagram of Emergency phone Recovery Framework

The Figure 3.2 also shows internal system activities including monitoring power-off attempts, detecting unauthorized access, activating stolen mode, locking the device, sharing live location, storing logs, and sending alerts. The framework interacts with the **Firestore Cloud Database** to store security settings, location details, alerts, and recovery data. Overall, the diagram provides a high-level view of system functionality and user interaction

3.3 REQUIRED MODULES/ LIBRARIES/ FRAMEWORKS

- **Android Studio:** Development environment used for implementing and testing the Emergency Phone Recovery Framework.
- **Kotlin:** Primary programming language used for Android application development and framework implementation.
- **Firebase Authentication:** Used for secure user authentication and login verification.
- **Firebase Realtime Database:** Cloud database used for synchronization, emergency data storage, and recovery information management.
- **Google Location Services (GPS API):** Used for location tracking and emergency recovery support.
- **Android SDK:** Provides Android libraries, APIs, emulator support, and application testing tools
- **XML:** Used for designing application layouts and user interface components.
- **Firebase Cloud Services:** Used for cloud synchronization, emergency alerts, and recovery-related communication.
- **Notification Manager:** Used for generating emergency alerts and recovery notifications to the owner.
- **Android Device Sensors:** Used for monitoring suspicious activities and supporting recovery operations.

3.4 METHODOLOGY AND PROCESS

3.4.1 USER AUTHENTICATION

The framework begins with secure user authentication using Firebase Authentication services. Users register and log into the application using authorized credentials. Authentication ensures that only legitimate users can access the Emergency Phone Recovery Framework and its recovery functionalities.

3.4.2 THREAT DETECTION PROCESS

During normal operation, the Threat Detection Module continuously monitors device behavior and user activities for suspicious actions such as unauthorized shutdown attempts, abnormal access behavior, or possible theft-related operations. The framework analyzes these activities to determine whether the behavior is normal or suspicious.

3.4.3 EMERGENCY MODE ACTIVATION

If suspicious activity is detected, the framework instantly activates Emergency Mode. In this stage, the application performs emergency actions such as device lockdown, unauthorized shutdown blocking, owner alert generation, and enabling location tracking support. These actions help prevent unauthorized access and improve recovery chances.

3.4.4 CLOUD & ALERT MANAGEMENT

The framework uses Firebase cloud services for synchronization, recovery management, and emergency communication. Recovery logs, device information, and alert data are securely stored and synchronized through Firebase Realtime Database. Emergency alerts and notifications are sent to assist the owner during theft situations

3.4.5 RECOVERY VERIFICATION MECHANISM

The system implements a secure Recovery Gate mechanism during Recovery Mode. Only the legitimate owner can regain access to the framework using a secure recovery password verification process. This layered security mechanism prevents unauthorized users from bypassing the recovery system

3.4.6 OWNER RECOVERY PROCESS

After successful ownership verification, the owner regains access to the virtual phone-level environment. The framework restores authorized access and allows the owner to manage recovery-related operations securely. This process demonstrates proactive smartphone protection and recovery support within application-level limitations

4.RESULTS

The developed Emergency Phone Recovery Framework was successfully implemented and tested on an Android device. The results demonstrate the functionality of the propose framework, including application lock setup, recovery password configuration, trusted email registration, device monitoring, stolen mode activation, live location tracking, recovery verification, and restoration of normal device operation. The following screenshots illustrate the working of each module and validate the effectiveness of the proposed recovery mechanism

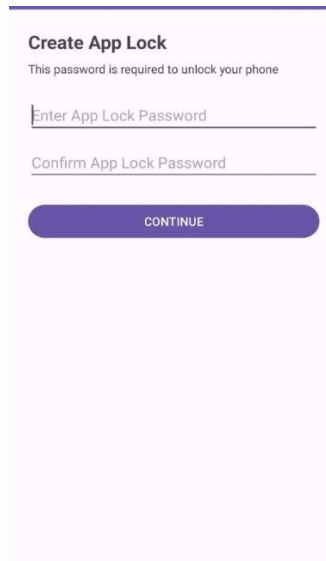
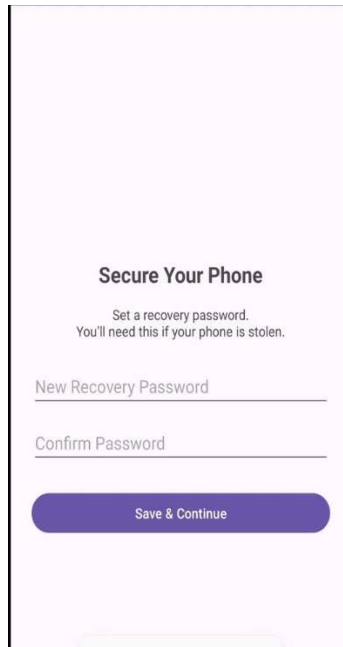


Figure 4.1 setup app lock password

The Fig 4.1 shows the App Lock Setup interface where the user creates a secure lock password similar to a smartphone screen lock for protecting the framework from unauthorized access

A mobile app screen with a light pink background. At the top, the title "Secure Your Phone" is centered in bold. Below it, a subtitle reads "Set a recovery password. You'll need this if your phone is stolen." There are two input fields: "New Recovery Password" and "Confirm Password". At the bottom, there is a purple rounded button labeled "Save & Continue".

Secure Your Phone

Set a recovery password.
You'll need this if your phone is stolen.

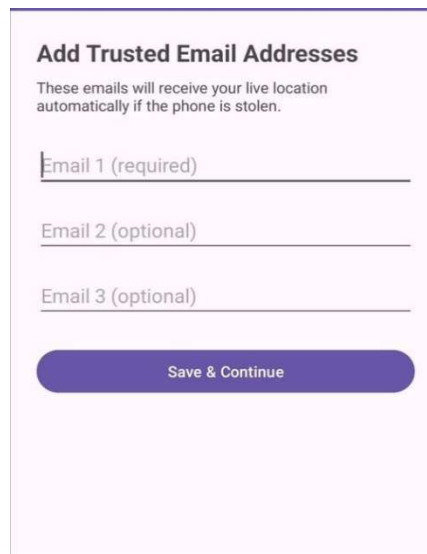
New Recovery Password

Confirm Password

Save & Continue

Figure 4.2 Recovery password setup

The Fig 4.2 shows the Recovery Password interface used during Recovery Mode. The legitimate owner must enter the secure recovery password to verify ownership and exit Recovery Mode

A mobile app screen with a light pink background. At the top, the title "Add Trusted Email Addresses" is centered in bold. Below it, a subtitle reads "These emails will receive your live location automatically if the phone is stolen." There are three input fields: "Email 1 (required)", "Email 2 (optional)", and "Email 3 (optional)". At the bottom, there is a purple rounded button labeled "Save & Continue".

Add Trusted Email Addresses

These emails will receive your live location
automatically if the phone is stolen.

Email 1 (required)

Email 2 (optional)

Email 3 (optional)

Save & Continue

Figure 4.3 Trusted Email Setup Interface

The Fig 4.3 shows the Trusted Email Setup interface where the user configures trusted email addresses during the initial framework setup. These email addresses are used to send emergency alerts and live location information during theft situations.



Figure 4.4 Physical Recovery Key

The Fig 4.4 shows the Physical Recovery Key used by the legitimate owner to recover access if the recovery password is forgotten.

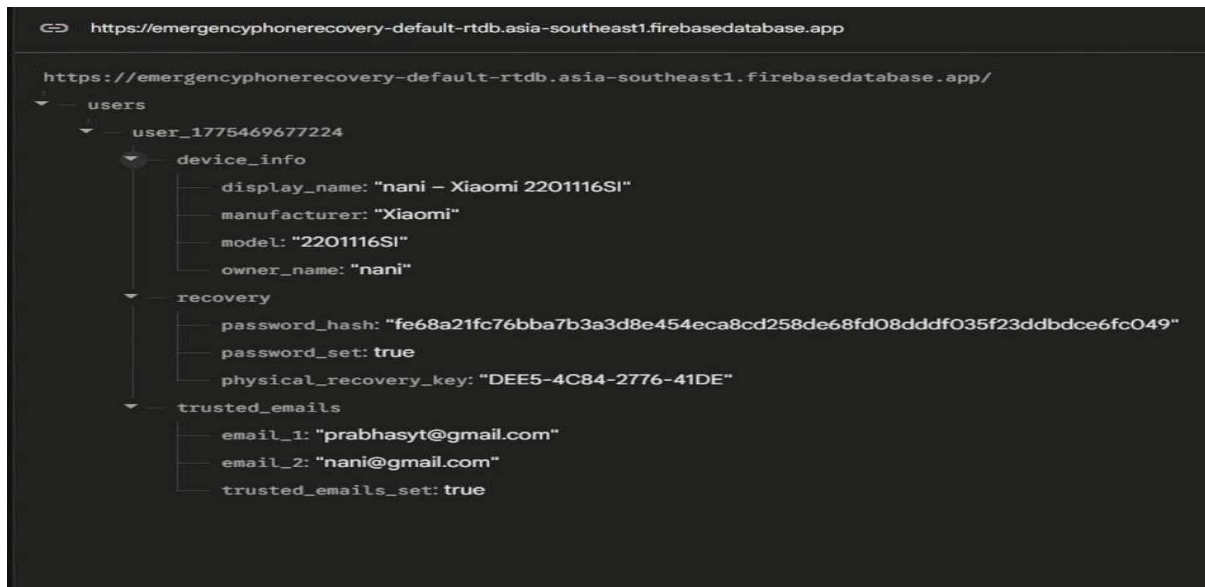


Figure 4.5 Firebase Realtime Database

The Fig 4.5 shows the Firebase Realtime Database used in the Emergency Phone Recovery Framework. The database stores and synchronizes application activities such as recovery information, trusted emails, triggered emergency mechanisms, location tracking data, and other recovery-related updates in real time.

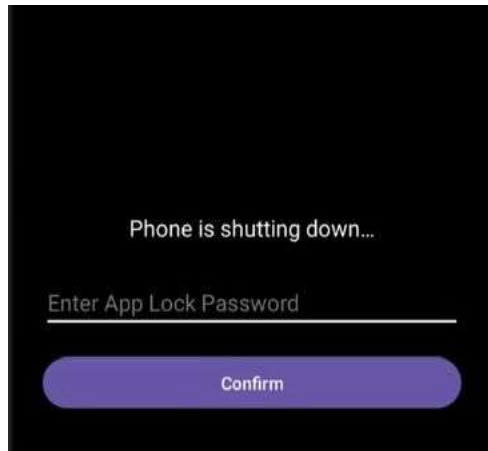


Figure 4.6 Shutdown Verification Screen

The Fig 4.6 shows the shutdown verification mechanism of the framework. The user must enter the correct app lock password to power off the phone. After five continuous wrong attempts, the framework automatically activates Stolen Mode for security protection

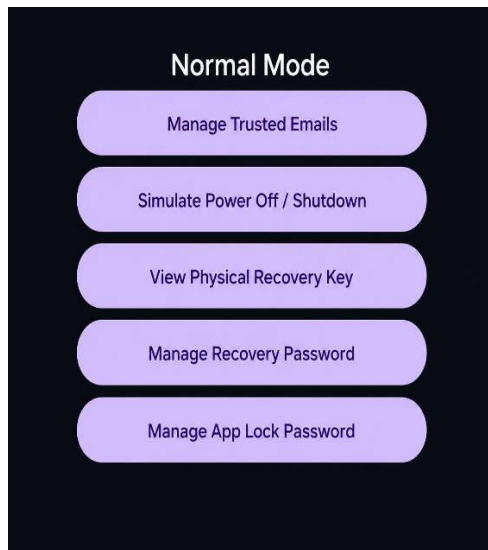


Figure 4.7 Normal mode Dashboard

The Fig 4.7 shows the main dashboard of the Emergency Phone Recovery Framework operating in Normal Mode. From this dashboard, the user can access and modify framework functionalities such as trusted emails, recovery password, physical recovery key, and shutdown simulation settings. All updates are synchronized with the Firebase Realtime Database.

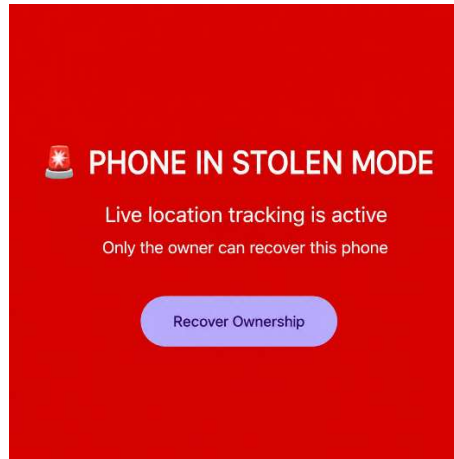


Figure 4.8 Stolen Mode Interface

The Fig 4.8 shows the Stolen Mode activated after five continuous wrong screen lock password attempts. In this mode, the phone becomes unusable until ownership is recovered. The framework automatically enables live location tracking, prevents phone shutdown, and sends emergency alerts to the trusted email addresses configured by the owner

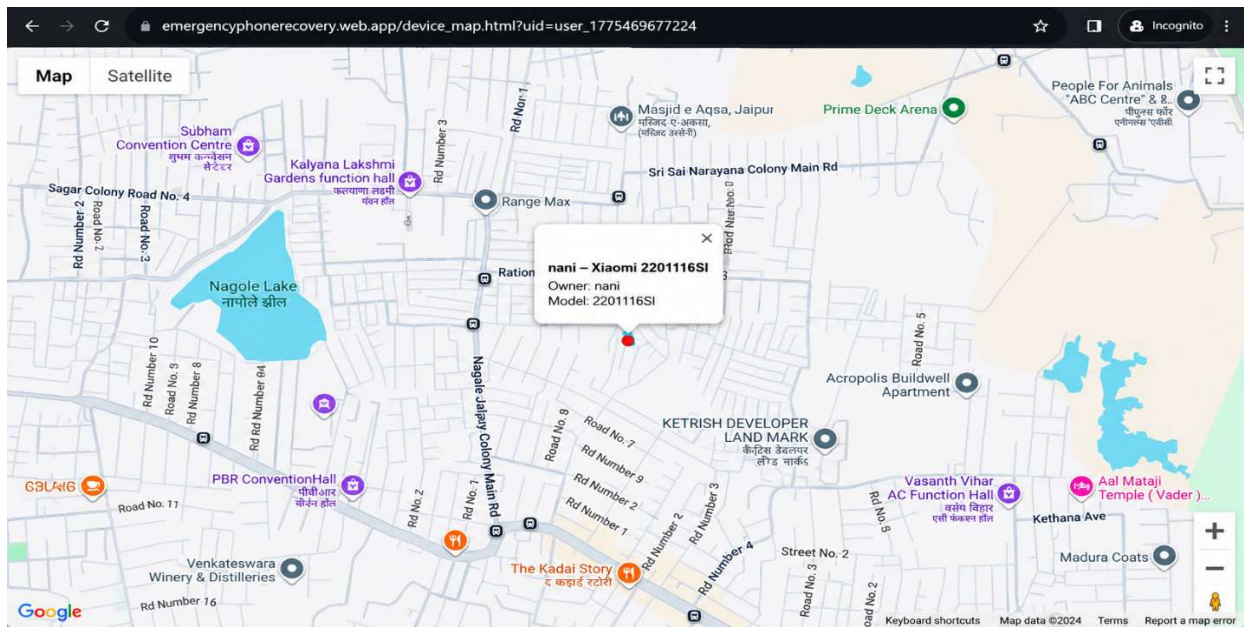


Figure 4.9 Live Location Tracking System

The Fig 4.9 shows the live location tracking system activated during Stolen Mode. When Stolen Mode is enabled, the framework displays the real-time location of the device on Google Maps to assist the owner in recovery.

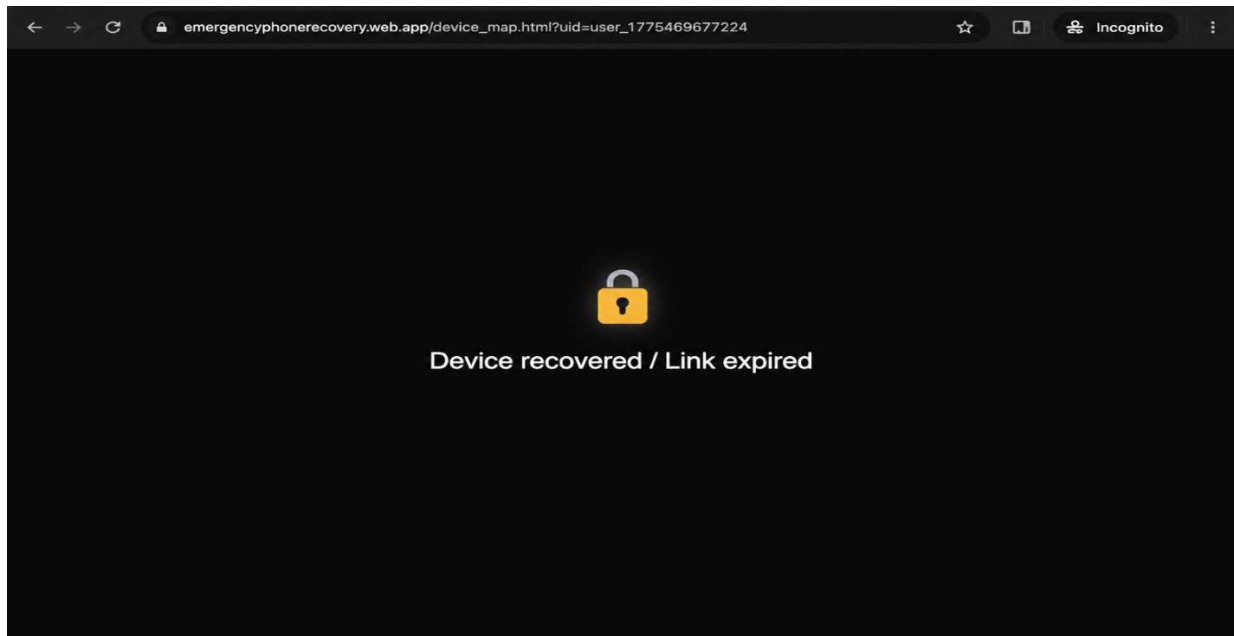


Figure 4.10 Expired Recovery Link

The Fig 4.10 shows the expired location-sharing webpage displayed after the phone is successfully recovered. Once Recovery Mode is completed, the live tracking link becomes inaccessible automatically for security protection.

A screenshot of a web form titled "Verify Recovery Password". The form has a light purple background. At the top, the title "Verify Recovery Password" is in bold. Below it is a password input field with a placeholder ".....1|". Under the input field is a purple button with the text "Verify". At the bottom of the form, there is a link that says "Forgot recovery password?".

Figure 4.11 Recovery Mode Interface

The Fig 4.11 shows the Recovery Mode interface where the user must enter the previously configured recovery password. After successful verification, the framework exits Stolen Mode and returns to Normal Mode; otherwise, the device remains in Stolen Mode.

4.12 Test Cases and Validation

Table 4.1 Test Cases Table of Emergency Phone Recovery Framework

Test Case ID	Test Description	Expected Result	Status
TC-01	App Lock Setup	Password saved successfully	Pass
TC-02	Recovery Password Setup	Recovery password created	Pass
TC-03	Trusted Email Registration	Email stored successfully	Pass
TC-04	Shutdown Verification	Verification screen displayed	Pass
TC-05	Wrong Password Entered 5 Times	Stolen Mode activated	Pass
TC-06	Live Location Tracking	Location displayed on map	Pass
TC-07	Email Alert Generation	Alert sent to trusted email	Pass
TC-08	Recovery Password Verification	Device recovered successfully	Pass
TC-09	Link Expiration After Recovery	Tracking link expired	Pass
TC-10	Update User Data in Application	Changes synchronized with Firebase Realtime Database	Pass

The proposed Emergency Phone Recovery Framework was tested under different operational scenarios. All modules functioned as expected, including password verification, stolen mode activation, live location tracking, trusted email alerts, and device recovery. The successful execution of all test cases validates the effectiveness and reliability of the proposed framework

5.CONCLUSION AND FUTURE SCOPE

5.1 CONCLUSION

The proposed Emergency Phone Recovery Framework successfully demonstrates a proactive smartphone security and recovery concept designed to improve protection against mobile theft and unauthorized access. The framework models the application as a virtual phone-level control environment capable of simulating advanced recovery and security mechanisms within Android application limitations. The system was implemented as a prototype using Android Studio, Kotlin, and Firebase for cloud synchronization and recovery management.

The framework successfully demonstrated features such as suspicious shutdown detection, Stolen Mode activation, live location tracking, trusted email alerts, Recovery Mode verification, and physical recovery key support. The layered recovery mechanism improved smartphone recovery support by restricting unauthorized access and allowing only the legitimate owner to recover the device.

Although the concept is platform independent, the current prototype implementation is demonstrated only for Android devices. Overall, the project successfully presents a practical prototype (Version 1) for proactive smartphone security and recovery assistance, demonstrating the feasibility of application-level smartphone protection and recovery mechanisms within Android platform limitations.

The developed framework was tested using multiple functional test cases covering app lock setup, recovery password verification, trusted email registration, stolen mode activation, live location tracking, Firebase synchronization, and device recovery operations. All implemented test cases were executed successfully, resulting in a functional success rate of 100%. The testing results validate the reliability and effectiveness of the proposed recovery framework under the defined operating conditions

5.2 FUTURE SCOPE

- **Hardware-Level Integration:** Integrate the framework with operating system and hardware-level security mechanisms for stronger smartphone protection and shutdown control.
- **Multi-User System Support:** Expand the framework to support multiple users, centralized management, and large-scale recovery operations.
- **Admin Dashboard Integration:** Develop a centralized admin dashboard for monitoring recovery activities, device status, alerts, and security management.
- **Cross-Platform Expansion:** Extend the framework beyond Android and support platform-independent smartphone recovery solutions for multiple operating systems.
- **Advanced Phone Hardening:** Implement stronger device hardening mechanisms to improve resistance against unauthorized access, forced shutdowns, and bypass attempts.
- **Cloud Analytics and Recovery Intelligence:** Develop intelligent monitoring and analytics mechanisms to analyze recovery events, suspicious activities, and user behavior patterns for improved theft detection and recovery decision support

BIBLIOGRAPHY

- [1] A. Parmar, K. Chandravanshi, S. Tiwari, V. D. Kola, M. Kumar, and S. Srivastava (2026) “Anti-theft Mobile Tracking Application”*
- [2] S. Roy, A. K. Shah, and U. Bhattacharya (2016) “Touch and Track: An Anti-theft and Data Protection Technique for Smartphones”*
- [3] M. E. Shalini and V. Shanti (2025) “A Novel Location-Based Anti-Theft Image Capture Application with Culpit Identification for Smartphones”*
- [4] N. Tasnim (2023) “Smartphone Loss Prevention System Using BLE and GPS Technology”*
- [5] G. Zheng, H. Zhang, Z. Wang, Y. Wei, and M. Dong (2019) “Mobile Phone Anti-theft Method Based on Mobile Track and User Characteristic”*
- [6] M. Z. Baik, M. M. Alqahtani, and A. Banjar (2025) “Detecting Theft over Pattern Change Recognition and Locating Stolen Micro Mobility Vehicles in Uncovered Signals Areas”*
- [7] M. L. Polla, F. Martinelli, and D. Sgandurra (2013) “A Survey on Security for Mobile Devices”*
- [8] Z. Muhammad, Z. Anwar, A. R. Javed, B. Saleem, S. Abbas, and T. R. Gadekallu (2023) “Smartphone Security and Privacy: A Survey on APTs, Sensor-Based Attacks, Side-Channel Attacks, Google Play Attacks, and Defenses”*
- [9] A. Rakhmadi, D. K. Aji, S. Rosad, A. Aziz, and R. Wahyusari (2025) “An IoT-Based Motorcycle Security System Using Arduino and Android for Theft Prevention”*
- [10] J. Landgren and C. Larsson (2024) “Proximity Based Mobile Device Theft Protection”*

APPENDIX

// Emergency Phone Recovery Framework – Key Code Snippets (Short Version)

// 1. Firebase Initialization

```
private lateinit var database: DatabaseReference
```

```
database = FirebaseDatabase.getInstance().reference
```

// 2. App Lock Password Verification

```
fun verifyAppLock(input: String, savedPassword: String): Boolean {  
    return input == savedPassword  
}
```

// 3. Wrong Attempt Counter

```
var wrongAttempts = 0
```

```
fun checkAttempts() {  
    wrongAttempts++  
  
    if (wrongAttempts >= 5) {  
        activateStolenMode()  
    }  
}
```

// 4. Activate Stolen Mode

```
fun activateStolenMode() {  
    StolenModeManager.setStolen(this, true)  
  
    startActivity(  
        Intent(this, StolenModeActivity::class.java)  
    )  
}
```

// 5. Recovery Password Verification

```
fun verifyRecoveryPassword(  
    enteredPassword: String,  
    savedPassword: String  
): Boolean {  
  
    return enteredPassword == savedPassword  
}
```

// 6. Recovery Mode Exit

```
fun recoverPhone() {  
  
    StolenModeManager.setStolen(this, false)
```

```

        startActivity(
            Intent(this, MainActivity::class.java)
        )
    }

// 7. Trusted Email Storage
fun saveTrustedEmail(email: String) {

    database.child("trusted_emails")
        .push()
        .setValue(email)
}

// 8. Live Location Tracking
fun updateLiveLocation(
    latitude: Double,
    longitude: Double
) {

    database.child("live_location")
        .setValue("$latitude,$longitude")
}

// 9. Emergency Alert Trigger
fun sendEmergencyAlert() {

    database.child("alerts")
        .push()
        .setValue("Stolen Mode Activated")
}

// 10. Shutdown Simulation
fun simulateShutdown() {

    if (verifyAppLock(
        enteredPassword,
        savedPassword
    )
    ) {

        finish()

    } else {

        checkAttempts()
    }
}

// 11. Physical Recovery Key

```

```

fun generateRecoveryKey(): String {

    return UUID.randomUUID()
        .toString()
        .substring(0, 8)
}

// 12. Firebase Realtime Updates
database.addValueEventListener(
    object : ValueEventListener {

        override fun onDataChange(
            snapshot: DataSnapshot
        ) {

            // Update app data
        }

        override fun onCancelled(
            error: DatabaseError
        ) {

        }

    }
)

// 13. Stolen Mode Check
if (StolenModeManager.isStolen(this)) {

    startActivity(
        Intent(
            this,
            StolenModeActivity::class.java
        )
    )
}

// End of Snippet

```